Pervasive and Mobile Computing 88 (2023) 101737

Contents lists available at ScienceDirect

Pervasive and Mobile Computing

journal homepage: www.elsevier.com/locate/pmc

Context-driven encrypted multimedia traffic classification on mobile devices☆

Mohammad A. Hoque [a,1], Benjamin Finley [a,*], Ashwin Rao [a], Abhishek Kumar [b,a], Pan Hui [e,d,a], Mostafa Ammar [c], Sasu Tarkoma [a,b]

[a] University of Helsinki, Helsinki, Finland
[b] University of Oulu, Oulu, Finland
[c] Georgia Institute of Technology, Atlanta, USA
[d] The Hong Kong University of Science and Technology, Hong Kong Special Administrative Region
[e] The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China

ARTICLE INFO

Article history:
Received 8 September 2022
Received in revised form 23 November 2022
Accepted 9 December 2022
Available online 10 December 2022

Keywords:
Encrypted traffic classification
Mobile context
Multimedia applications
Mobile components

ABSTRACT

The Internet has been experiencing immense growth in multimedia traffic from mobile devices. The increase in traffic presents many challenges to user-centric networks, network operators, and service providers. Foremost among these challenges is the inability of networks to determine the types of encrypted traffic and thus the level of network service the traffic needs to maintain an acceptable quality of experience. Therefore, end devices are a natural fit for performing traffic classification since end devices have more contextual information about device usage and traffic. This paper proposes a novel approach that classifies multimedia traffic types produced and consumed on mobile devices. The technique relies on a mobile device's detection of its *multimedia context* characterized by its utilization of different media input/output (I/O) components, e.g., camera, microphone, and speaker. We develop an algorithm, MediaSense, which senses the states of multiple I/O components and identifies the specific multimedia context of a mobile device in real-time. We demonstrate that MediaSense classifies encrypted multimedia traffic in real-time as accurately as deep learning approaches and with even better generalizability.
© 2022 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Mobile devices have been generating 60% of all Internet traffic, and a significant portion of this traffic comes from multimedia applications that involve streaming and interactive video and audio. Along with the content from popular content providers, user-generated content is also on the rise. In 2017, the Cisco Visual Network Index predicted that traffic from various services such as video broadcast, live streaming, augmented reality (AR), and virtual reality (VR) applications would grow five-to-seven fold by 2022 [1]. This growth was aggravated during the corona pandemic that required millions of people to engage in remote work, interactive online education, and consuming entertainment at home [2].

☆ The work is an extension of a paper published in the proceedings of PerCom 2022.

* Corresponding author.
E-mail addresses: mohammad.a.hoque@helsinki.fi (M.A. Hoque), benjamin.finley@helsinki.fi (B. Finley), ashwin.rao@helsinki.fi (A. Rao), abhishek.kumar@oulu.fi (A. Kumar), panhui@ust.hk (P. Hui), ammar@cc.gatech.edu (M. Ammar), sasu.tarkoma@helsinki.fi (S. Tarkoma).

[1] This work was done when Mohammad Hoque was employed at the University of Helsinki.

https://doi.org/10.1016/j.pmcj.2022.101737
1574-1192/© 2022 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

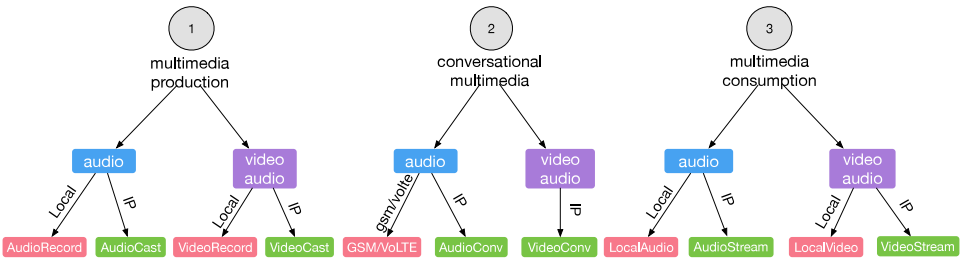


Fig. 1. Multimedia interaction types on mobile devices. Our devices can produce multimedia, consume multimedia, or use multimedia during conversations. The second layer denotes the typical content types produced, consumed, or exchanged during the interaction. The third layer represents the context according to the multimedia type and their medium of use: 'local' implies on-device media production and consumption, while 'IP' denotes the use of IP for multimedia production and/or consumption.

While ubiquitous devices enable a diverse set of multimedia activities [3], users generally have limited control over their traffic after it leaves their device. Consequently, a new set of personalized or user-centric networking applications are emerging, such as Personal Virtual Network (PVN) [4] and Middle Box Zero (MBZ) [5]. These solutions perform traffic inspection [6], and monitor network performance [7] on mobile devices. Some approaches also rely on root certificates to perform deep packet inspection on encrypted packets [6,8] or certificate pinning to perform similar inspection [9,10]. Naturally, such approaches incur significant privacy concerns. Furthermore, they currently lack a privacy-aware traffic classification mechanism to assist in performing networking activities, such as performance monitoring, protecting privacy, and requesting quality of service (QoS). Alternatively, deep learning algorithms that learn very application-specific features, such as Deep Packet [11] leverages application signatures from the initial Secure Sockets Layer (SSL[2]) and Transport Layer Security (TLS) packets. However, they require large training datasets and the classification and training requires significant energy making them sub-optimal for mobile devices. Furthermore, an application can be responsible for different types of traffic: a video conferencing solution can suggest users to switch off the video when the network connectivity is poor, thus resulting in drastically different network traffic characteristics. This problem is aggravated by the increasing prevalence of Domain Name System (DNS[3]) over TLS which is aimed at eliminating opportunities for eavesdropping and in-network modifications of DNS queries and responses [12]. Therefore, we need an energy-efficient, accurate, and privacy-aware traffic classification mechanism on mobile devices.

In this article, we detail our approach that classifies multimedia traffic into specific multimedia activity categories (such as streaming, broadcasting, and conversation) using a set of general (non app-specific) features. Our approach leverages on a mechanism to identify such multimedia activities, which we call *multimedia contexts*. PVNs or MBZs can employ our approach to detect multimedia traffic types and then perform various optimizations (such as network selection, performance monitoring for different traffic types, traffic padding for preserving privacy, or route optimization for improved QoS) in a more privacy-preserving fashion.

A device's multimedia context describes whether the device is used for producing content, consuming content, or conversing (thus both producing and consuming content). We present a unique sensing algorithm, MediaSense, to accurately detect *eleven* such multimedia contexts of a device. MediaSense can be used to identify various multimedia traffic scenarios in real-time on mobile devices, and it relies on the answers to the following questions:

(i) what are the content types users interact with?;
(ii) how do users interact with each type of content?;
(iii) which I/O components are utilized during such interactions on smart devices; and
(iv) what are the states of these I/O components while interacting with different content types?

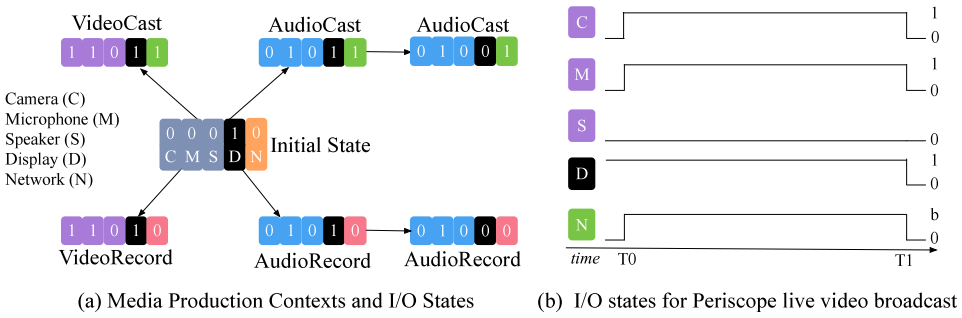
Our two key contributions are as follows.

(1) Context Definition and MediaSense. To the best of our knowledge, we are the first to (a) define multimedia contexts, and (b) propose a method to detect and use such contexts on mobile devices. We study *sixty-two* popular multimedia applications on Android and iOS devices and classify them according to how users interact with different multimedia contents using these applications (Section 2). We use our analysis to define *eleven* multimedia contexts. These contexts can be abstracted into three high-level multimedia contexts: (i) multimedia production, (ii) multimedia consumption, and (iii) conversational multimedia, as demonstrated in Fig. 1.

Next, we explore how these contexts use several media I/O hardware components on mobile devices and present a multimedia context sensing algorithm called MediaSense. Through an extensive evaluation using over *62* applications,

2 SSL and TLS are widely adopted cryptographic protocols used by many internet applications.

3 DNS is an internet naming system that, for example, translates human-readable domain names (such as google.com) to the actual routable internet protocol (IP) addresses of servers.



(a) Media Production Contexts and I/O States (b) I/O states for Periscope live video broadcast

Fig. 2. The media production contexts and the corresponding I/O states. (a) AudioRecord and VideoRecord refer to local (on-device) recording of audio and video, whereas AudioCast and VideoCast refer to live audio and video broadcast from a device using a microphone and camera. (b) States of the I/O components while broadcasting live with Periscope.

we demonstrate that with flow-level information MediaSense identifies the correct multimedia contexts with 97%–100% accuracy. Furthermore, it identifies the corresponding voice/video over IP, live broadcast, and multimedia streaming network flows in real-time with an accuracy higher than 93% (Section 4) with negligible energy.

(2) Comparison with state-of-the-art approaches We further evaluate the performance of state-of-the-art deep learning approaches, such as 1D/2D Convolutional Neural Networks (CNNs) [11,13,14], for encrypted multimedia traffic classification (Section 5). We capture the network traffic of the target multimedia applications and label them according to six IP-based multimedia contexts presented in Fig. 1 and train the CNNs. Our evaluation shows that these approaches perform poorly or have inadequate generalization performance (e.g., to new applications in a multimedia context). In contrast, MediaSense is generic across different multimedia types and for new apps, and very energy efficient.

The rest of the article is organized as follows. We detail the contexts of various multimedia applications and their usage of I/O components in Section 2. MediaSense is presented and evaluated in Sections 3 and 4 respectively. Section 5 investigates the performance of CNNs for encrypted multimedia traffic classification and compares with MediaSense. Section 6 highlights the potential use cases of MediaSense, and the related works are discussed in Section 7. The paper concludes in Section 8.

2. Multimedia applications and contexts

In this section, we detail how an application's use of I/O components on mobile devices can be leveraged for describing multimedia contexts. For our analysis, we focus on the following I/O components: camera, microphone, speaker, display, and the network. Across all the multimedia applications we observe that the state of these I/O components can be leveraged to identify the multimedia context. We investigate the utilization of these I/O components by *sixty two* multimedia applications (see Tables 2, 3, and 4) on Android and iOS devices; we used Nexus 6, LG G5, and iPhone 6/6s for this study. Of the 62 applications, 35 are available on both Android and iOS devices, 17 are only available for Android devices, and 10 are only available for iOS devices.

All the required I/O components are typically initialized simultaneously depending on the application's characteristics. Since the user needs to launch an app via the touch screen, it is intuitive that the screen is busy when the application is launched. Therefore, the initial states of the I/O components for all the multimedia applications are the same: the camera is not in use, the microphone is not in use, the speakers are not in use, the display is in use, and the network is not in use. In Table 1 and Figs. 2–4, we represent the status of the I/O components with '1' and '0', where the bits represent the busy and free status of the corresponding I/O components. For instance, a 1 for camera implies that at least one of the cameras is being used by the application, while a 0 implies that the application is not using the cameras. In practice, a free I/O component can be physically off, e.g., as with a display. In contrast, the network I/O status does not represent the status of the network interface. Rather the status depends on network activities, such as the bit rates of the applications in terms of sending, receiving, or both.

We do not assume that only one application is running at any given time. In Section 2.4 we detail mixed multi-contexts, i.e., scenarios where multiple applications use the same resources; we argue that such mixed contexts continue to be quite rare because they tend to affect the quality of experience.

2.1. Multimedia production contexts

A mobile device is in a production context when an application records audio/video on local storage or broadcasts live to remote consumers. Voice Memos and Camera are the default audio and video production applications on iOS devices. Similarly, Android devices have built-in microphone and camera applications. In addition to these applications, Mixlr, Periscope, and StreamLab are popular live audio and video broadcasting applications as presented in Table 2. By

Table 1
Multimedia context. The state of the I/O determines the multimedia context. Note that a change in state does not always imply a change in context. For instance, a device can enter the Audio Cast context with the display turned on (0 1 0 1 1), and continue to be in that context even after the display is switched off (0 1 0 0 1).

C	M	S	D	N	Context
0	0	0	0	0	N-A (Background process)
0	0	0	0	1	N-A (Background process)
0	0	0	1	0	N-A (Phone display active)
0	0	0	1	1	N-A (Non multimedia app/service)
0	0	1	0	0	Media consumption (Local audio)
0	0	1	0	1	Media consumption (Audio Stream)
0	0	1	1	0	Media consumption (Local audio)
0	0	1	1	1	Media consumption (Audio Stream)
0	1	0	0	0	Media consumption (Audio record)
0	1	0	0	1	Media consumption (Audio cast)
0	1	0	1	0	Media consumption (Audio Record)
0	1	0	1	1	Media consumption (Audio cast)
0	1	1	0	0	Media production (Audio recording and playback)
0	1	1	0	1	Conversation (Audio Conversation)
0	1	1	1	0	Media production (Audio recording and playback)
0	1	1	1	1	Conversation (Audio/Video conversation)
1	0	0	0	0	Media production (Local image capture)
1	0	0	0	1	Media production (Video cast)
1	0	0	1	0	Media production (Local image capture)
1	0	0	1	1	Media production (Video cast)
1	0	1	0	0	Media production (Local image capture)
1	0	1	0	1	Media production (Video stream)
1	0	1	1	0	Media production (Local image capture)
1	0	1	1	1	Media production (Video stream)
1	1	0	0	0	Media production (Video record)
1	1	0	0	1	Media production (Video Cast)
1	1	0	1	0	Media consumption (Video record)
1	1	0	1	1	Media production (Video cast)
1	1	1	0	0	Media production (Video record)
1	1	1	0	1	Conversation (Audio/Video conversation)
1	1	1	1	0	Media production (Video record)
1	1	1	1	1	Conversation (Audio/Video conversation)

Table 2
Multimedia production contexts and the corresponding 19 applications for Android (a) and iOS (i) devices. The *Periscope* service has been discontinued from March 2021.

AudioRecord	Voice Memes(i), Voice Recorder(a), Dolby On(a), Hi-Q Recorder(a), RecForge II(a), ASR Recorder(a), Wear Recorder(a).
AudioCast	Mixlr(a/i), Spreaker(a/i).
VideoRecord	Open Camera(a), Camera(i), Dolby On(a), HD Camera(a), Camera MX(a), Camera360(a).
VideoCast	Periscope(a/i), StreamLab(a/i), BroadcastMe(a/i), Facebook Live(a/i).

Table 3
32 applications/services of four multimedia consumption contexts for Android (a) and iOS (i) devices.

LocalAudio	Vox(i), Flacbox(i), Radsone(i), jetAudio(i), Stezza(i), Music Player Go(a), Poweramp(a), Omnia(a), Pulsar(a), VLC(a), AIMP(a).
AudioStream	Spotify(a/i), TuneIn(a/i), Tidal(a/i), qobuz(a/i), Idagio(a/i), ShoutCast(a/i), Soundcloud(a/i).
LocalVideo	VLC(a/i), MX Player(a/i), PlayerXtreme(a/i), KMPlayer (a/i), OPlayer Lite(i), 8Player (i).
VideoStream	YouTube (a/i), Vimeo(a/i), Dailymotion(a/i), HBO(a/i), Netflix(a/i), Twitch(a/i), Prime Video (a/i), Periscope(a/i).

Table 4
Conversational contexts and 11 applications on Android (a) and iOS (i) devices. Only VoLTE/GSM calls are responsible for non-IP AudioConv context.

AudioConv	WhatsApp(a/i), IMO(a/i), Viber(a/i), Kakao Talk(a/i), Line(a/i), Skype(a/i), Messenger(a/i), Duo(a/i), VoLTE/GSM(a/i), Snapchat (a/i).
VideoConv	WhatsApp(a/i), IMO(a/i), Viber(a/i), Kakao Talk(a/i), Line(a/i), Skype(a/i), Messenger(a/i), Duo(a/i), Snapchat (a/i), FaceTime (i).

investigating these applications, we derive four media production contexts emerging from the initial state as shown in Fig. 2(a).

Firstly since video recording typically requires users' attention, applications primarily remain in the foreground while recording so the user can monitor the video in real-time.[4] If the user switches to another app or turns off the display,

[4] Even in cases where video recording applications do not stay in the foreground, only Android allows display-off recording and such recording is not supported in mainstream apps (due to privacy concerns). Therefore, such recording is not a major use case.

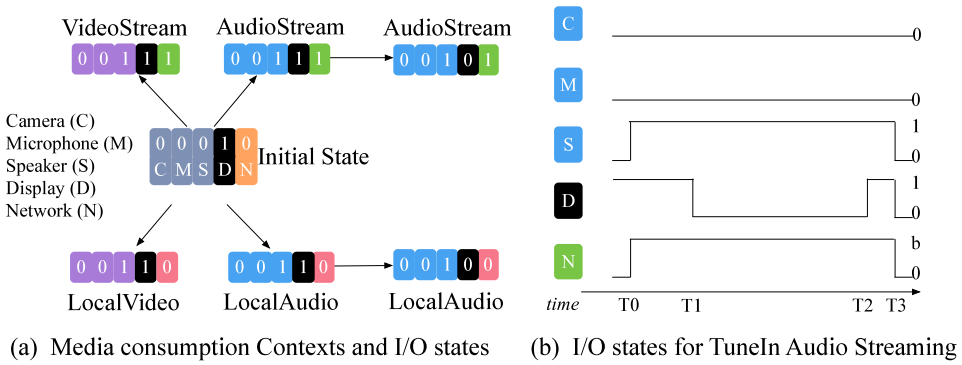


Fig. 3. The media consumption contexts and the states of the I/O components. (a) LocalAudio and LocalVideo refer to audio and video playback from the local (on-device) storage, whereas AudioStream and Video Stream refer to streaming audio and video from remote services. (b) States of the I/O components while streaming audio with TuneIn.

the camera and microphone typically become free. In contrast, audio recordings and live audio broadcasts more often continue in the background. Thus, the states of I/O components for the recording applications differ from those of the live broadcasting applications only by the network (as they do not transmit).

Fig. 2(b) shows the states of the I/O components during a live video broadcasting session of Periscope. We observe that the live broadcast begins at $T0$, and the application initializes the camera and microphones. The output component, the display, is also used, and data transmission begins. The broadcasting terminates at $T1$. Periscope also initiates transport control protocol (TCP) connections. We observed that the uplink bit rates of Periscope and Mixlr are 459 kbps and 128 kbps for broadcasting live video and audio, respectively.

2.2. Multimedia consumption contexts

A mobile device is in a consumption context when an end-user plays multimedia content from local storage or streams from a remote service provider.

Both Android and iOS devices have default applications for audio/video playback from local storage. In addition, we investigated popular streaming applications, such as YouTube, TuneIn, Periscope live streaming, and many others presented in Table 3. Through this exploration, we derive four media consumption contexts, as shown in Fig. 3(a). For watching videos from local storage or video streaming, display and speaker are mandatory; i.e., the playback stops when the user switches to another application. In contrast, audio applications can still play audio while in the background. Furthermore, streaming applications download content from a remote server and thus require IP connectivity. Despite the diverse set of multimedia streaming applications, such as on-demand streaming and live/pseudo-live streaming, we observe that audio and video consumption exhibit the same I/O states. Overall the media consumption applications vary significantly: some apps play with negligible initial playback delay, whereas some continue to cache and depend on the user's input.

Fig. 3(b) shows that only speaker and network activities begin when a TuneIn audio streaming session starts at $T0$. The display is turned off at $T1$ and turned on again at $T2$. Finally, the user terminates the streaming session at $T3$. TuneIn initiates multiple TCP flows as soon as the playback begins. The bitrates of the TuneIn audio streams vary from 24 kbps to 320 kbps. We also observe that the downlink bit rate of Periscope, i.e., the bit rate when viewing a Periscope live stream of other Periscope users, is similar to the uplink bit rate. Along with the application-specific bit rates, we also observed different ON/OFF patterns in the network traffic [15].

2.3. Conversational multimedia contexts

A device is in a conversation context when a user engages in an audiovisual conversation with another remote user using a conversational application.

In addition to GSM and VoLTE calls, we also experiment with the popular VoIP applications presented in Table 4. Fig. 4(a) shows that the states of the I/O components change according to the conversation types. An audio conversation does not need the display to be active during the call. In contrast, a video call uses all the media I/O components.

Conversational applications have two media contexts, i.e., audio or video conversations. Fig. 4(b) demonstrates that a WhatsApp VoIP call begins at $T0$, and all the I/O components become busy, except the camera. The display turns off at $T1$ and turns on again at $T2$. Finally, the call terminates at $T3$. WhatsApp initiates both TCP and User Datagram Protocol (UDP) flows as soon as the call begins. The TCP flows are mostly used for signaling, and UDP flows carry the media. The bitrate of the audio flow in each direction ranges from 17–22 kbps, and the bitrate increases to a few hundred kbps during video conversations.

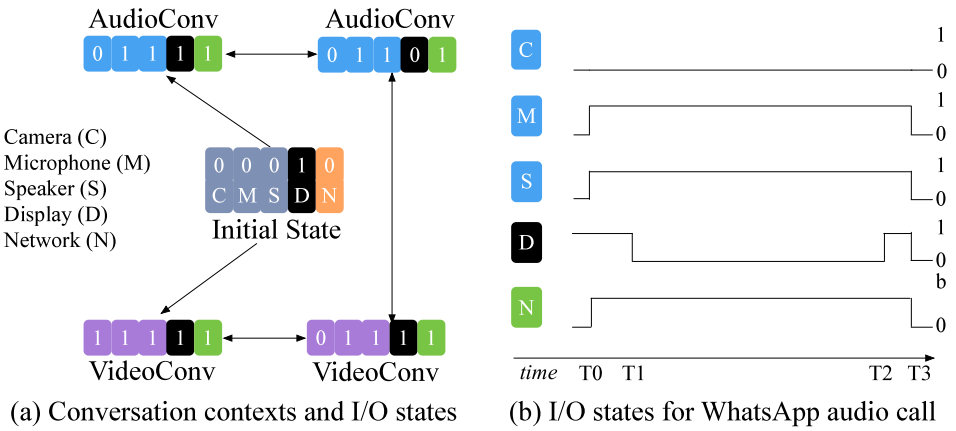


Fig. 4. The conversation contexts and the states of the corresponding I/O components. (a) AudioConv and VideoConv represent Audio and Video call contexts respectively. (b) States of the I/O components during a WhatsApp audio conversation.

A video conversation can also proceed without an active camera on either the caller's or callee's device. When the caller initiates a video call, all the media sensors are activated on the callee's device. If the caller turns off the camera after the call is established, the user still needs to keep the display active, as the caller's device receives video from the other end. When both users turn off their cameras, the media context changes to an audio call (Fig. 4).

2.4. Mixed multimedia contexts

A mixed multimedia context is a context wherein multiple apps produce multimedia contexts simultaneously (e.g., using Skype (Video Conv) while also watching a YouTube video (Video Stream)). Such contexts are, in theory, limited because multiple applications cannot utilize some I/O components simultaneously. For example, on Android, multiple applications cannot use the same (physical or logical) camera simultaneously.

However, multiple applications can use the same display, speaker, and, in limited circumstances, microphone[5] simultaneously. Thus, many types of mixed multimedia contexts are, in practice, possible. We experimentally validate this by testing the simultaneous use of several different types of multimedia apps on Android 13 (using a Pixel 6a). We were able to validate all mixed context combinations except for those in which two apps try to use the same or even different logical cameras at the same time (e.g., a Video Conv on the front camera and a Video Cast on the rear camera). Thus, Android devices still have some limitations in terms of mixed contexts.

Generally, we estimate that mixed contexts are still quite rare because they have significant downsides such as needing to use split screen mode. Thus using a single app designed for these more complex use cases is more likely (as the experience can be better tailored than trying to use several apps simultaneously). For completeness, we note that on iOS multiple applications cannot use the same camera[6] or microphone simultaneously but can use the same display or speaker simultaneously. Thus in general a similar though somewhat more restrictive logic applies to iOS devices.

The main justification for limiting the simultaneous use of a microphone or camera is privacy. Specifically, users may inadvertently disclose private information if, for example, they are not aware that a background app continues using a component even after switching to a different foreground multimedia app.

2.5. Summary

In this section, we have investigated many multimedia applications for mobile devices. We have shown that the utilization of the I/O components by multimedia applications can be generalized across both Android and iOS devices. This requirement of I/O components also allows us to extend the generalization to an arbitrary number of applications for media consumption, production, or conversation.

At the first level (Fig. 1), the distinct usage of the microphone and speaker components differentiates the production, conversational, and consumption contexts. While at the second level, the camera separates video from audio contexts for production and conversational media, whereas the display separates video from audio for media consumption. All the contexts are separated at the third level according to network activities, i.e., transmitting, receiving, or exchanging traffic.

5 See https://developer.android.com/guide/topics/media/sharing-audio-input.

6 See https://developer.apple.com/documentation/avkit/accessing_the_camera_while_multitasking.

Algorithm 1: MediaSense

```
▷ Comment 1: Pre-computed features.;
mediaFeatures = Map(mediaContext, bitrate);
while true do
    trafficstat = getTXRXbytes();
    mic = sampleMicrophone() ∈ {1,0};
    speaker = sampleSpeaker() ∈ {1,0};
    camera = sampleCamera() ∈ {1,0};
    display = sampleDisplay()∈ {1,0};
    ▷ Comment 2: Audio/Video media contexts.
    mediaContext = camera|mic|speaker|display;
    media = camera|mic|speaker;
    T_media = gettimeoftheday();
    ▷ Comment 3: IP-based media contexts
    if (mediaContext(!network)) then
        ▷ Comment 3.1: Compute network features based on network stats and other I/O
        mediaVec = computeFeatures(trafficstat, mediaContext);
        ▷ Comment 3.2: Lookup the conditions (e.g., thresholds) for considering context as using the network
        mediaFet = getConditions(mediaContext, mediaFeatures);
        ▷ Comment 3.3: If the features meet the conditions then consider an IP-based media context
        if (mediaVec ≈ mediaFet) then
            mediaContext = mediaContext|network;
        end
    end
    ▷ Comment 4: Updating Video to Audio consumption.
    if ((mediaContext == VideoStream)&&(!display)) then
        mediaContext = mediaContext|(!display);
    end
    ▷ Comment 5: Voice/Video call state changes.
    if ((mediaContext == VideoConf)&&(!camera)) then
        mediaContext = mediaContext|(!camera);
    end
    if (media==0) then
        ▷ Comment 6: MediaContext duration.
        Media_session = gettimeoftheday() − T_media
    end
end
```

Note that augmenting a phone with a headset via Bluetooth or cable does not affect the state of the I/O components; it only changes the route of the audio signal. Thus, being outdoors, indoors, or mobile does not change the need for these I/O components. This also applies to adjusting brightness, camera focus, or adjusting volume. However, a loss of signal or poor signal may disrupt a VoIP call, streaming session, or live broadcast and may terminate the media context.

3. MediaSense

Given that a user interacts with an arbitrary multimedia application, we devise MediaSense (Algorithm 1) that scans the states of five I/O components to infer the resulting multimedia context. We implement MediaSense as a user-level service for Android devices. The service runs as a background service and looks for multimedia contexts periodically at 1 Hz. Whenever one or more I/O components changes states, MediaSense initiates a new multimedia context. The algorithm first checks whether the media context is audio or video-related with the camera and display (Comment 2 in Algorithm 1). Then, the algorithm checks the traffic flow statistics in the uplink and downlink to separate IP-based contexts from local media (Comment 3 in Algorithm 1). We describe these steps in detail below.

3.1. Separating audio/video contexts

The algorithm periodically scans the states of the I/O components using on-device system application programming interfaces (APIs) to determine the media context.

(1) Conversational Multimedia Context. Fortunately, Android provides APIs for applications to indicate their modes of operation to the AudioManager [16], thus allowing other applications to query the status of the AudioManager via getMode() API. AudioManager operates in one of three modes; IN_CALL, IN_COMMUNICATION, and RINGTONE. These modes indirectly indicate that an ongoing context is conversational and that both the microphone and speaker are busy. TelephonyManager has getCallState() to indicate a GSM/VoLTE call [17] and thus expresses the states of the microphone and speaker. Table 5 summarizes the mapping between Android APIs and the corresponding media I/O components. MediaSense characterizes a context as a video conversation if the AudioManager is in one of the modes and one of the cameras is initialized at the same time. MediaSense implements registerAvailabilityCallback() from CameraManager [18] to poll camera status, i.e., available/unavailable, exactly when the audio mode changes.

Table 5
Android APIs for detecting media contexts and utilization of Media I/Os.

Android API	I/O component	User permission
AudioManger.getMode()	Microphone, Speaker	No
AudioManager.getMode(), CameraManager. registerAvailabilityCallBack()	Camera, Microphone, Speaker	No
AudioManager.isMusicActive()	Speaker	No
CameraManager. registerAvailabilityCallBack()	Camera, Microphone	No
MediaRecorder.record()	Microphone	Yes

Table 6
Features considered for identifying the media contexts. faststartbyte denotes the amount of data downloaded during the first 10 s of streaming and up and down rate are bitrates.

Up rate	Down rate	if Features then Context
✓	X	if (Bitrate ≥8 kbps, microphone) then AudioCast else AudioRecord
✓	X	if (Bitrate ≥8 kbps, microphone, camera) then VideoCast else VideoRecord
✓	✓	if (Bitrate ≥8 kbps, microphone, speaker) then AudioConv else GSM/LTE
✓	✓	if (Bitrate ≥8 kbps, microphone, speaker, camera) then VideoConv
X	✓	if (faststartbytes ≥100 kB, Bitrate >300 kbps, speaker, display) then VideoStream else LocalVideo
X	✓	if (faststartbytes ≥100 kB, Bitrate <300 kbps, speaker, !display) then AudioStream else LocalAudio

(2) Multimedia Consumption Contexts. The isMusicActive() API from AudioManager helps to differentiate music playback contexts from VoIP/GSM calls or other media production contexts on the device. This API provides the speaker information. However, the API does not differentiate whether the playback is audio or video. In other words, it could be an audio-only application or a video application that uses the speaker for the audio track. We also could not find APIs hinting about streaming. In Fig. 3, we notice that distinguishing between audio and video consumption contexts is not straightforward, given just the statuses of the I/O components. The reason is that audio applications require only speakers and can work while keeping the display active or inactive. Therefore, the algorithm first assumes a media consumption context is video type. Then when the display turns off, the media context is changed to audio type as the media session continues.

(3) Multimedia Production Contexts. MediaSense uses APIs which do not require explicit user permission to detect the earlier described media contexts. Similarly, the algorithm uses registerAvailabilityCallback() from CameraManager to detect the video production contexts from the Camera or Periscope like applications. This API initializes the camera and microphone together. However, detecting the state of the mic is not possible without explicit user permission. MediaSense implements MediaRecorder APIs with user permission to detect the audio production contexts due to the applications presented in Table 2.

3.2. Separating local/IP-based media contexts

MediaSense distinguishes IP-based contexts from the local media context by identifying the traffic flows from a set of live flows using flow-level information, as presented in Table 6. It relies on the Android virtual private network (VPN) API to gain access to such information from live flows. There is no other feasible way to access network traffic information on Android mobile devices. This API was also used by Mopeye [7] and AntMonitor [19]. Unlike these, MediaSense neither installs root certificates nor performs deep packet inspection. The algorithm uses five tuples as the flow identifier. MediaSense does not compute all the features in the table for a media context. It rather computes media context-specific features. These features are derived from our observations in Section 2 with the following reasoning.

(1) Conversational Multimedia Contexts. Unlike the other applications, conversational traffic carries voice or video data in both directions. Voice traffic has a minimum bit rate of 14 kbps. However, the bitrates of these applications depend on the underlying codec [20], which can be as low as 8 kbps in one direction [21]. A GSM/VoLTE call be identified using the getMode() API and by noting that there is no uplink and downlink traffic (see Table 6).

(2) Multimedia Consumption Contexts. On-demand streaming applications, e.g., YouTube, Spotify, begin streaming with a fast start phase and download 10–40 s of equivalent playback content. Spotify streams audio via persistent hypertext transfer protocol (HTTP) connections over TCP, regardless of the device type [22]. The audio streams are encoded at 96–360 kbps, and the selection depends on the subscription type. The size of the first segment is 139.53 kB [22]. YouTube downloads more than one megabyte during the fast start phase. Periscope downloads content at a constant bit rate after the fast start. Therefore, MediaSense relies on isMusic() API and the flow features presented in Table 6 to separate streaming contexts from local music playback.

(3) Multimedia Production Contexts. Periscope's outgoing bitrate is 459 kbps. A 64 kbps outgoing bitrate is very common for live audio broadcasting. Mixlr supports 32–128 kbps. We consider a lower bound of 8 kbps as the context feature. Overall MediaSense uses the bitrate feature along with the CameraManager & MediaRecorder APIs to separate the IP-based based contexts from the local recording contexts.

Table 7
MediaSense correctly identified 100% (all true positives) of multimedia sessions and network flows for audio/video broadcasts during the first 10 s of broadcasts.

VideoCast				AudioCast			
App	TP/#sess	TP/#flow	Bitrate-mbps	App	TP/#sess	TP/#flow	Bitrate-kbps
Periscope	53/53	53/53	0.4–0.7	Mixlr	76/76	76/76	16–128
Streamlabs	39/39	39/39	0.5–1.5	Spreaker	20/20	20/20	16–128
BroadcastMe	26/26	26/26	0.2–0.7	–	–	–	–
FacebookLive	29/29	29/29	0.6–1.4	–	–	–	–

4. Performance evaluation

We installed MediaSense on an LG G5 (Android 8.0) with an LTE connection for evaluation. The data plan allowed a maximum of 45 Mbps and 20 Mbps speed for downlink and uplink, respectively. The device had 101 applications installed, including 62 multimedia apps. However, we interacted with only one multimedia application at a time, and the session duration remained between 30–60 s. We denote this as a media session or the duration of a media context. The device was fully charged during the experiments to avoid any system-assisted performance degradation [23].

We evaluate MediaSenseon how accurately it can differentiate local versus IP-based media contexts in the media sessions and how accurately it can identify the corresponding network flows. We estimate flow detection accuracy as:

$$Accuracy = (TP + TN)/(TP + TN + FP + FN) \tag{1}$$

We define the proportions in the equation as follows. True positive (TP) denotes the number of cases correctly identified as IP-based contexts. False positive (FP) denotes the number of cases incorrectly identified as IP-based contexts. True negative (TN) denotes the number of cases correctly identified as local contexts. And False negative (FN) denotes the number of cases incorrectly identified as local contexts.

The accuracy in identifying an IP-based context is expressed by the precision, which estimates the proportion of true positives in the corresponding sessions. This can be stated as:

$$Precision = TP/(TP + FN) \tag{2}$$

In contrast, the recall expresses the local context accuracy, which estimates the proportion of true negatives in the IP-based context that resemble local media contexts. This can be stated as:

$$Recall = TN/(TN + FP) \tag{3}$$

For network flow identification or traffic classification performance, we only use the *accuracy* measure.

4.1. Multimedia production contexts

We first experiment with the multimedia production applications presented in Table 2.

Local/non-IP production context identification. There are 220 sessions for 11 local media production applications. MediaSense did not identify any media network flows during these local media production contexts. In other words, MediaSense correctly identified all the AudioRecord and VideoRecord sessions without any FNs, and therefore, the recall is 100%.

IP-based production context identification. We experimented with six applications. The bitrates of the audio broadcasts varied between 16 and 128 kbps. Periscope and BroadcastMe had the average bitrates for medium-quality videos, whereas Streamlabs and Facebook Live transmitted high-definition videos. Network traffic should be generated by the broadcasting applications when the media context is initiated or later on based on user interaction with the application. MediaSense correctly identified all 243 media contexts as presented in Table 7. Consequently, there were no FPs while detecting the broadcast contexts, and the precision is 100% for both AudioCast and VideoCast.

Traffic Classification. The very high precision in detecting IP-based media contexts (see Table 7) also indicates that MediaSense detects the relevant flows. However, it can generate TPs (detecting the actual flow), FPs (detecting a wrong flow), or FNs (not detecting a flow). Note that TNs are not possible for flow detection. Therefore, we use Eq. (1) to determine the flow detection accuracy. Table 7 shows that MediaSense identified *243* audio/video live broadcast flows during *243* sessions of Mixlr, Spreaker, Periscope, Streamlabs, BroadcastMe, and Facebook Live. There were no FP or FN, and MediaSense is 100% accurate in identifying the broadcasting flows.

4.2. Multimedia consumption contexts

Media consumption contexts relate to live streaming, pseudo-live streaming, or on-demand streaming applications and other local playback applications (see Table 3). In the streaming cases, since the content is first downloaded and then played, there is an initial playback delay of 3–5 s [15]. Therefore MediaSense considers this absolute time difference,

Table 8
MediaSense's performance in identifying VideoStream contexts and network flows in real-time.

VideoStream						
App	Sessions		Flows			
	TP	FN	TP	FP	FN	Bitrate-mbps
YouTubeLive	49	3	52	0	0	0.35–4.7
Periscope	48	4	47	5	0	0.2–3.01
Twitch	53	0	53	0	0	0.6–6.3
Vimeo	47	0	47	0	0	0.4–6.2
Dailymotion	42	3	38	5	0	0.35–11
Netflix	43	0	43	0	0	3.5–29
Prime Video	36	0	36	0	0	0.35–4.7
HBO Nordic	29	0	29	0	0	0.5–9.1

Table 9
MediaSense's performance in identifying AudioStream contexts and network flows in real-time (display-off).

AudioStream						
App	Sessions		Flows			
	TP	FN	TP	FP	FN	Bitrate-mbps
TuneIn	52	0	47	5	0	0.03–0.44
ShoutCast	43	0	38	5	0	0.03–0.44
Qobuz	50	0	49	1	0	0.6–4.1
Idago	39	0	37	2	0	0.5–5.0
Tidal	33	0	31	2	0	1.2–5.2
Spotify	29	0	26	3	0	2.5–7.5
SounCloud	23	0	22	1	0	0.8–4.5

i.e., $|t_{media} - t_{flow}|$, in filtering the flows. In addition to display status, we also consider 16–300 kbps bitrates to indicate audio streams and higher bitrates to indicate video streams. The absence of network flows with such features indicates a local media consumption context.

Local/non-IP context identification. We did not observe FNs for the local audio and video playbacks from the ten applications, as MediaSense associates the flow initiation time with context beginning and considers the flow bitrates. Therefore, it identifies 200 local audio/video playback contexts from 10 applications with 100% recall.

IP-based video consumption context and Traffic Classification. Table 8 shows that MediaSense correctly identified 97% of 357 VideoStream contexts from eight applications. Three YouTube, four Periscope, and three Dailymotion sessions were identified as AudioStreams, as the bitrates were below 300 kbps. Likewise, MediaSense correctly detected 95% of the multimedia flows during VideoStream contexts as shown in Table 8. However, MediaSense incorrectly detected additional flows in ten sessions (FPs), including during Dailymotion and Periscope contexts. MediaSense also revealed that Twitch, Vimeo, and Prime Video use multiple flows to download content. More than 70% of the 48 Twitch sessions and all the 47 Vimeo sessions had two flows, and 36 Prime sessions had four flows per session. Table 8 shows that MediaSense identified their network flows with 97% accuracy.

IP-based audio consumption context and Traffic Classification. Table 9 shows that MediaSense identified 269 AudioStream contexts from seven applications with 100% accuracy as there were no FNs. However, it identified some of the corresponding network flows as FPs. The FPs occurred due to unrelated background flows during streaming. Likewise, MediaSense detected the corresponding network flows with 93% accuracy due to some false positives resulting from background flows.

4.3. Conversational multimedia contexts

We investigated the performance of MediaSense with all the conversational applications in Table 4. Since the delay requirement of conversation multimedia is very strict, MediaSense considers the flows initiated within three seconds of the media contexts, i.e., $|t_{media} - t_{flow}| \leq 3$ s.

Local/non-IP context identification. Conversational contexts have data exchange in both directions. However, GSM and VoLTE data do not follow the same path as normal application data. Therefore, the absence of flows with at least eight kbps bitrates in both directions indicates an ongoing GSM/VoLTE call. On LG-G5, the algorithm identified 20 conversational sessions due to 20 GSM calls without any FN; thus, the recall is 100%.

IP-based context identification and Traffic Classification. MediaSense identified 329 AudioConv and 329 VideoConv contexts with similar features with 100% precision on LG G5, as shown in Tables 10 and 11. There were no FPs. Although there could be rate control by these applications [24,25], an 8 kbps bitrate (and an active camera) are sufficient for

Table 10
MediaSense's performance in identifying VideoConv contexts and network flows in real-time.

VideoConv						
App	Sessions		Flows			
	TP	FN	TP	FP	FN	Bitrate-mbps
WhatsApp	50	0	50	0	0	0.3–0.51
Skype	50	0	50	0	0	0.4–0.8
Viber	50	0	50	0	0	0.9–2.2
IMO	50	0	50	0	0	0.3–0.7
Kakao	28	0	28	0	0	0.3–0.5
Duo	26	0	26	0	0	0.35–1.8
Messenger	25	0	25	0	0	0.4–1.1
Snapchat	25	0	25	0	0	0.3–0.7
Line	25	0	25	0	0	0.3–0.7

Table 11
MediaSense's performance in identifying AudioConv contexts and network flows in real-time.

AudioConv						
App	Sessions		Flows			
	TP	FN	TP	FP	FN	Bitrate-kbps
WhatsApp	50	0	50	0	0	10–23
Skype	50	0	50	0	0	43–73
Viber	50	0	50	0	0	10–18
IMO	50	0	50	0	0	14–18
Kakao	22	0	22	0	0	10–24
Duo	26	0	26	0	0	40–55
Messenger	23	0	23	0	0	8–23
Snapchat	25	0	25	0	0	14–18
Line	25	0	25	0	0	10–40

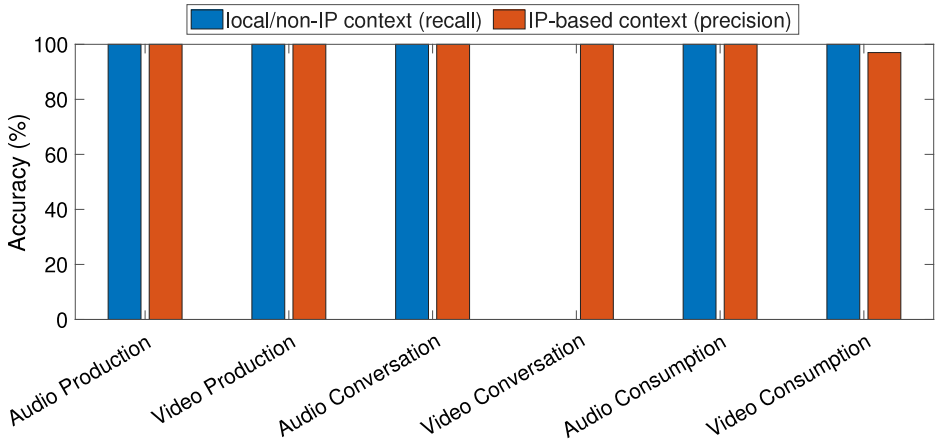


Fig. 5. MediaSense performance in identifying the 11 multimedia contexts from Fig. 1. Note that the local/non-IP context for audio conversation represents GSM and VoIP calls.

detecting video calls and flows. A conversational context can switch to a hybrid mode when the camera is off/on during a conversation. Consequently, MediaSense also accurately identifies such hybrid contexts via camera status and flow bitrate features. Similarly to the audio/video broadcasting flows, MediaSense identified 658 conversational flows with 100% accuracy without any FP or FN (see Tables 10 and 11) (see Fig. 5).

4.4. Energy consumption

We further measured the electric current consumption of MediaSense on the Nexus 6. Although Power Monitor [26] would provide better estimates, modern mobile devices come with difficult-to-access batteries, and thus, instrumenting these devices is very challenging. We instrumented MediaSense with Android API to sample the run time current

Table 12
The average current consumption of Nexus 6 over the periods 60 s for the multimedia applications.

Application	Context	Baseline (mA)	MediaSense (mA)
Voice recorder	AudioRecord	110	119
Camera	VideoRecord	700	800
Mixlr	AudioCast	140	139
BroadcastMe	VideoCast	439	440
TuneIn	AudioStream	140	200
YouTubeLive	VideoStream	525	540
WhatsApp	AudioConv	222	231
Phone	AudioConv(GSM)	135	155
WhatsApp	VideoConv	743	700
VLC	LocalVideo	545	579
VLC	LocalAudio	130	122

consumption at 1 Hz and tested on the Nexus 6. The device had 101 applications installed, including the 62 multimedia applications connected to a WiFi access point. The device was fully charged during the measurements. MediaSense consumes 70 mA on average while the device is idle. Table 12 compares the average current consumption of the Nexus 6 for nine applications with eleven media contexts in the absence (baseline) and presence of MediaSense. During the video contexts, the display was ON, and the front camera was used for the VideoConv and VideoCast contexts. The display was off during the audio contexts. MediaSense computes flow statistics, tracks media contexts, and associates contexts with the flows. We notice MediaSense does not consume considerably more energy compared to the baseline. Though we also note that this energy consumption analysis is primarily a sanity check as we perform only a few measurements over a short period (60 s). Therefore inherent variations on this timescale mean in a few cases we measure MediaSenseas using less energy than the baseline, however this is likely not the case. We plan to perform a more comprehensive energy analysis in future work.

5. Performance comparison

In this section, we compare the performance of MediaSense with state-of-the-art deep learning methods for traffic classification [11,13,14]. We note that since these deep learning methods are for network traffic classification we can only compare performance for the six IP-based contexts as the five local contexts do not produce network traffic (by definition).

5.1. Deep learning approaches

We evaluate the performance of session and flow-level 1D and 2D Convolution Neural Networks (CNN)s and a packet-level 1D-CNN to classify encrypted multimedia traffic. The session and flow-level 1D-CNN models have two 1D convolution layers with 32 and 64 filters, respectively. Each convolution layer is followed by a 1D max-pooling and terminated by two fully connected layers. The session and flow-level 2D-CNN model is constructed by replacing 1D convolution and pooling layers with the corresponding 2D layers. Further details of the models can be found in [13,14]. In contrast, the packet-level 1D-CNN has two 1D convolution layers also each followed by 1D max-pooling and a final three fully connected layers. Further details of this model can be found in [11]. These deep models train on actual byte data (as we discuss further), thus flow, or packet-level feature engineering is not required.

5.2. Dataset & training the networks

The total size of the training dataset is 6.7 GB in *pcap* format. It contains the traces for the subsets of applications of six IP-based classes (contexts) discussed in the previous sections. A traffic session is defined by a 5-tuple (source IP, source port, destination IP, destination port, and transport protocol). A flow is similar and considers traffic direction (so the IP and port are not reversible).

We use 80% of total flows or sessions for the flow and session-level models to train the 1D and 2D-CNN models and the remainder for testing. Each flow or session is represented (to the model) by the first 784 bytes of the pcap file containing only that session or flow (as in the original model [13]) and thus includes the first few packets of the flow or session along with packet metadata (which is part of the pcap file format) such as their capture timestamps.

Whereas for the packet-level model, we also use 80% of the packets for training and the remainder for testing. Each packet is represented (to the model) by the first 1500 bytes of the packet (with zero-padding or truncation if necessary). We note that some packets are larger than 1500 bytes due to capturing before TCP segmentation offload, large segment offload, and generic segment offload features of network interfaces [27]. Additionally, we need to consider the high-class imbalance due to the significant differences in packet volumes for different multimedia classes (e.g., video vs. audio). Therefore, we perform random undersampling of the training data to equalize the class frequencies. In contrast, the testing

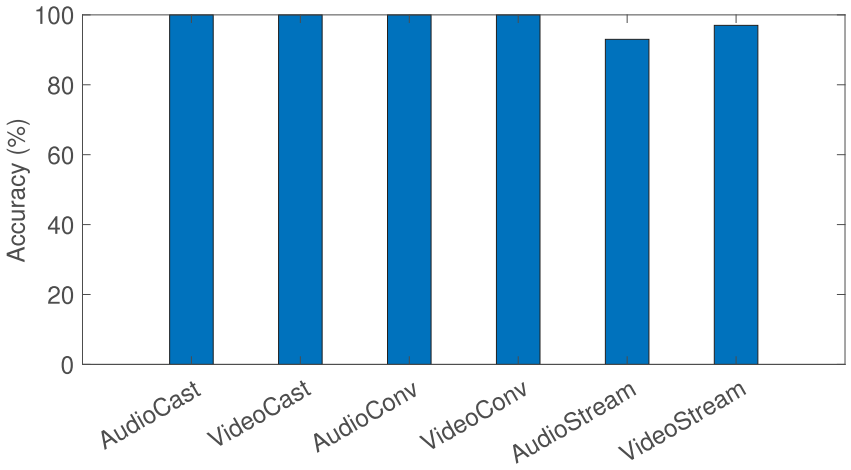


Fig. 6. Real-time flow classification performance of MediaSense for different IP-based multimedia contexts.

dataset is not undersampled to maintain a realistic evaluation. Finally, we provide the complete confusion matrix results, as only accuracy values can be misleading.

For all models, we additionally test model generalization concerning new apps in categories by testing with different apps in the training data and the testing data (e.g., for video streaming, we have Netflix and Vimeo in the training data and YouTube in the testing data). In this additional test, for the session-level model, we also mask the IP address due to concerns that this can unfairly imply or leak app identity [14].

We train the CNN models using Tensorflow for the flow and session-level models and PyTorch for the packet level model on a Linux server with an Nvidia Tesla K80 GPU. In terms of metrics, we train the flow and session-level models for about 30 epochs until the performance levels, and similarly, we train the packet-level model for five epochs.

5.3. Evaluation

Figs. 7(a), and 7(b) illustrates the test confusion matrices for the 1D-CNN session and packet-level model respectively. We omit the flow-level and 2D-CNN model results because their performance is similar to those presented in the figure. The results show that all the deep learning approaches achieve reasonable accuracy (whether on a flow or packet level). However, the session-level methods have difficulty distinguishing VideoCast from AudioCast sessions as both contexts resemble constant bitrate traffic and have similar packet header attributes. In contrast, the packet-level approach faces difficulty distinguishing AudioStream sessions from AudioCast sessions.

Furthermore, in Fig. 8, we find that the generalization performance for both deep learning models is significantly worse compared to the base evaluations (from Figs. 7(a) and 7(b)) and MediaSense. Specifically, we find that the accuracies of 1D-CNN session-level model for categories are 76%, 79%, 30%, 39%, 67%, and 34% (compared to 89%, 96%, 79%, 56%, 89%, 95% for the base evaluation) for a new application. Likewise, the 1D-CNN packet-level model the accuracies are 95%, 55%, 95%, 60%, 80%, and 88% (compared to 98%, 97%, 49%, 100%, 97%, 100% for the base evaluation). This suggests that these models are learning app-specific features, for example from the first few bytes of the SSL/TLS exchange, rather than the more general category-specific features of multimedia traffic. Ref. [14] also notes similar findings for the packet-level model.

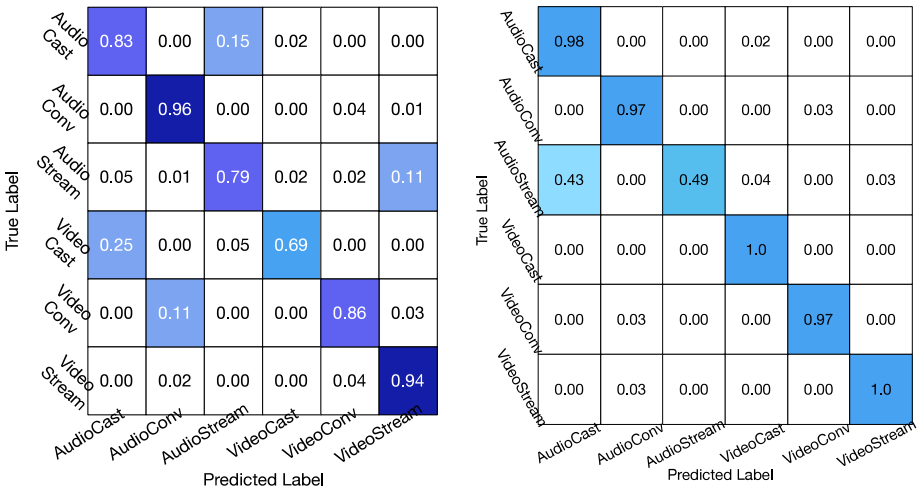
Next, we estimate a few of these app-specific features for these deep learning models. Specifically, we focus on the packet-level model and apply several different explainable AI (eAI) algorithms to determine which parts of the packets (i.e., parts of the header and payload) are most important for classification. We use the Captum eAI library (over PyTorch) and apply the integrated gradients, deeplift, and saliency methods to the trained[7] model [28]. We then calculate the mean attribution values for each byte (of the 1500 bytes) over the testing dataset across the three methods.

We find that the most important bytes (all in the top ten) include those dealing with TCP ports, TCP options (such as the TCP option kind and timestamp[8]) and TLS length. This further suggests that changes in the networking aspects of these apps (for example during app updates) will significantly impact classification performance.

Overall, by determining and associating contexts, MediaSense identifies such traffic with similar or higher accuracy and better generalization (for new apps) on mobile devices without any training as demonstrated in Fig. 6, and with negligible energy cost.

[7] Specifically the model trained over our original training dataset, not the generalization training dataset.

[8] We note that these bytes are important even if the specific timestamps are set to zero during preprocessing.



(a) 1D-CNN (Flow) (b) 1D-CNN (Packet)

Fig. 7. Confusion matrices for session and packet level 1D-CNN models.

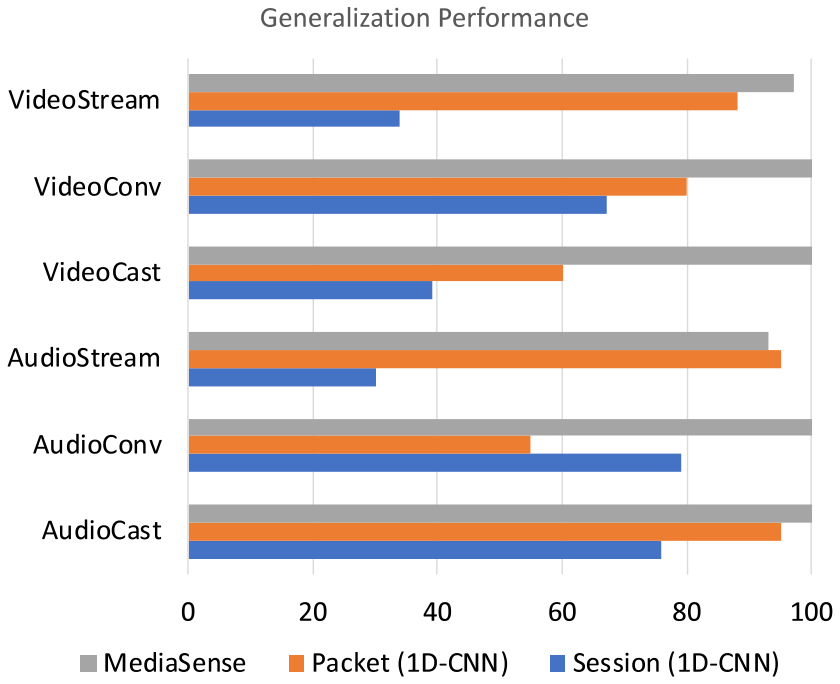


Fig. 8. Generalization performance of different models.

6. Discussion

There are many potential uses of information about a device's multimedia context, and in this section we discuss some implications of our work and the potential avenues for future work.

In Section 6.1, we discuss how MediaSense can be used on non-mobile devices such as smart televisions. We build on this discussion in Section 6.2, where we discuss MediaSense and augmented reality (AR), virtual reality (VR) and extended reality (XR) applications and services. As discussed in Section 2.4, our algorithm is largely agnostic to the available APIs. However, we do acknowledge that the implementation of MediaSense will be governed by the APIs provided by the operating systems running on these devices and this constitutes a limitation of MediaSense. In Section 6.3, we discuss how MediaSense can be used for on-device optimizations such as network selection and smart energy consumption.

Finally, in 6.4, we discuss how MediaSense can be leveraged by the different stakeholders controlling and managing the networks used by our devices.

6.1. MediaSense on non-mobile devices

In this work, we demonstrated how multimedia context information could be used to classify encrypted multimedia traffic in real-time on mobile devices. Desktop computers, laptops, and other handheld devices have similar I/O components. Therefore, our methodology for building MediaSense could potentially be extended to support such devices as well. The key challenge will be to identify the usage of the I/O devices during multimedia production and consumption and to find analogous APIs for their respective OSes.[9] Furthermore, mixed multimedia contexts might be more prevalent given the potentially different sizes of these devices (thus removing some of the downsides to split screen mode) and limitations (like the potential to have different apps use cameras simultaneously).

6.2. Media context and extended reality applications

Recent advances in sensor technologies have enabled a new breed of applications that operate in three-dimensional space: AR, VR, and mixed reality (MR). 360-degree/volumetric videos streamed by these devices have stringent QoS requirements in terms of latency and bandwidth which current mobile networks may not be able to support [29]. MediaSense can aid in optimized resource allocation for these applications. For example, in a VR-based collaborative gaming application, players communicate over VoIP, thereby using a speaker and microphone. Though, mobile headsets like Oculus Quest 2 are also equipped with many additional sensors (than available on traditional mobile devices), such as depth sensors which MediaSense does not currently consider and which may require different QoS than RGB cameras. Thus this is a current limitation of MediaSense.

In terms of XR, meetings are likely to be done with holographic techniques, where the participants are present through their holographs, which gives the impression that everyone is present in the same room. MediaSense should already classify such sessions as video conversations but new contexts (e.g., holographic conversation) and corresponding bitrate thresholds may be required due to required QoS differences.

6.3. Leveraging MediaSense for on-device optimizations

Mobile multimedia contexts can be used by mobile devices for on-device optimization of application performance, device performance, and energy consumption.

Intelligent network selection. Most devices have multi-homing capabilities, i.e., they are capable of using multiple communication technologies concurrently. For instance, smart phones can concurrently use cellular and Wi-Fi networks, and each of the connected networks can offer a wide range of network quality. A mobile network can offer 5G and 4G connectivity to its clients, and Wi-Fi network can offer connectivity over a family of IEEE 802.11 protocols. Identifying the mobile multimedia context, can enable the mobile device to select a network based on the network capabilities and the demands of the applications. Specifically, on-device traffic conditioning according to signal strength and multimedia context can improve the quality of experience.

Smart energy consumption. The multimedia context can also provide mobile operating systems hints on the usage of computation, storage, and networking resources. For instance, video streaming is more compute-intensive than audio streaming. The CPU governor can therefore plug/unplug the cores and scale CPU frequency according to the context resulting in energy savings. Similarly, the devices can schedule tasks according to the media context. Specifically, user applications can also use such contexts for energy-aware scheduling of background traffic [30,31].

6.4. Network management

Different stakeholders in the networks also can benefit from MediaSense. In 5G, the access networks classify traffic and send the QoS policies to mobile devices to apply [32]. The policies include dropping packets, routing packets according to the addresses, and marking flows with Differentiated Service Code Points (DSCPs). We believe that MediaSense is a practical approach that can be leveraged to extend DiffServ on mobile devices to complement the QoS architecture in 5G. As a result, the network operators do not have to perform multimedia traffic classification, as the traffic can be marked even before arriving at the access network. Furthermore, the operators can map the DSCP marked traffic to core or radio network slices [33] or leverage such information in determining the QoS Identifier of the 5G QoS flow (which can contain multiple network flows).

Similarly, Multiple Access Management Services (MAMS) framework enables networks and devices to negotiate uplink and downlink network paths based on the application needs and the characteristics and available resources on different

[9] Though, for example, in some cases the move towards unified mobile and desktop APIs such as with iOS and MacOS means even the same (or very similar) API calls might work. Furthermore, we believe that our algorithm is agnostic to the underlying APIs.

network connections [34]. Thus MAMS can leverage MediaSense to identify the application needs. Furthermore, network service providers can further use such information for billing, network planning/provisioning, and security.

Finally, as mentioned, MediaSense could be useful to user-centric networking solutions. These solutions allow personalizable on-device network traffic shaping (thus acting before traffic actually leaves the device). A prominent example of these solutions is Middle Box Zero (MBZ). MBZ leverages the VPN API (similar to MediaSense) to act as an on-device middlebox (allowing services such as a user-defined firewall, fine-grained traffic routing, and network troubleshooting).

MBZ could leverage MediaSense as a multimedia traffic classifier (thus avoiding the aforementioned privacy intrusive methods). Therefore, users could personalize the shaping of their different types of multimedia traffic (by, for example, routing different media contexts through different network interfaces). Specifically, in terms of integration, MBZ has a plugin architecture thus allowing a custom MediaSense plugin that could both classify traffic and subsequently perform the multimedia traffic shaping. In the Android case, each MBZ plugin is essentially just an Android activity with an associated GUI layout and native c++ code (typically for the traffic shaping code). We plan to investigate such integration in future work.

7. Related work

Identifying multimedia traffic is essential. Significant works have been done that are most suitable for network service providers to manage their networks. Different traffic classification methods can be divided into three categories.

Deep Packet Inspection. The MIMIC system [35] looks into HTTP logs of the adaptive streaming requests to estimate the average bitrates, bitrate switch, and the playback buffer status. Similarly, BUFFEST [36] investigates the HTTP logs. Lumen [8], and VPNGuard [6] investigate the encrypted packet payloads with the help of a VPN service and using custom root certificates. Several approaches investigated the codec formats of encrypted VoIP packets from Skype [37]. However, modern multimedia services, such as Periscope, and Netflix, communicate over HTTPS [38]. Therefore, the traditional port-based classification techniques do not work, and deep packet inspection [39] is difficult given the encryption.

Statistical Methods. Statistical methods rely on flow features, such as packet size distribution, packet gap, burstiness, and packet headers [40]. These features can be used to understand VoIP applications' traffic patterns, such as Skype [41]. Bonfiglio et al. [37] first looked into the statistical properties of message content and then matched with the Skype voice traffic sources by using Naive Bayesian techniques. Do and Branch used inter-packet gaps to classify VoIP traffic in real-time [42]. However, the flow features can vary with speech codecs and ambient noise [43].

Machine Learning. We have already evaluated two deep learning approaches in Section 5 using packet [11] and flow [13,14] level features. Some recent studies identified YouTube videos of different qualities from the encrypted traffic by modeling the relationship between burstiness, i.e., chunk size and gaps, and videos' quality [44]. The basic flow features can be used by machine learning algorithms, such as K-nearest neighbor clustering, to classify encrypted multimedia traffic [45]. Kim et al. [46] showed that Support Vector Machine achieves more than 98% accuracy with less training data than other machine learning algorithms.

Summary. The deep packet inspection methods are difficult on encrypted traffic. The various machine learning approaches require retraining the models, large training data, and energy consumption as the traffic pattern changes. In contrast, MediaSense is specifically for user-centric networks on end-devices. It performs much better with the help of media contexts. MediaSense is as accurate as of the existing deep learning approaches as demonstrated in Section 5.

8. Conclusions

This article introduces the multimedia context concept and presents a novel algorithm to identify the corresponding encrypted multimedia traffic on mobile devices in real-time. MediaSense computes and leverages the media context-specific flow features for finding and identifying the corresponding multimedia flows. The approach is energy efficient and can generalize across multimedia applications without training. MediaSense is also privacy-preserving, as it neither infers the application nor examines the actual packet data nor leaks the contexts. MediaSense opens the door for context-aware system and traffic optimization on mobile devices.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The work was supported by the Academy of Finland, Finland IDEA-MILL project (Grant Number 335934), Academy of Finland, Finland 5GEAR project (Grant Number 319669), Nokia Foundation, Finland Grant, Walter Ahlström Foundation, Finland Grant, Academy of Finland, Finland FIT project (Grant Number 325570), Academy of Finland, Finland 6G Flagship Program (Grant Number 346208), and GP INDFICORE, Finland EFFICACY Menot (Grant Number 24650101115). Mostafa Ammar's work was partially supported by NSF, USA grant NETS (Grant Number 1909040). In addition, the authors wish to thank the Finnish Computing Competence Infrastructure (FCCI) for supporting this project with computational and data storage resources.

References

[1] Cisco, Cisco Visual Networking Index: Forecast and Methodology, 2016—2021, Tech. rep., 2017.
[2] A. Feldmann, et al., The lockdown effect: Implications of the COVID-19 pandemic on internet traffic, in: Proceedings of the ACM Internet Measurement Conference, IMC '20, ACM, New York, NY, USA, 2020, pp. 1–18, http://dx.doi.org/10.1145/3419394.3423658.
[3] M.A. Hoque, A. Rao, A. Kumar, M. Ammar, P. Hui, S. Tarkoma, Sensing multimedia contexts on mobile devices, in: Proceedings of the 30th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video, NOSSDAV '20, ACM, New York, NY, USA, 2020, pp. 40–46, http://dx.doi.org/10.1145/3386290.3396935.
[4] D. Choffnes, A case for personal virtual networks, in: HotNets '16, ACM, New York, NY, USA, 2016, pp. 8–14, http://dx.doi.org/10.1145/3005745.3005753.
[5] J. Newman, A. Razaghpanah, N. Vallina-Rodriguez, F.E. Bustamante, M. Allman, D. Perino, A. Finamore, Back in control – An extensible middle-box on your phone, 2020, CoRR.
[6] Y. Song, U. Hengartner, PrivacyGuard: A VPN-based platform to detect information leakage on android devices, in: Proceedings of the 5th Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices, SPSM '15, ACM, New York, NY, USA, 2015, pp. 15–26, http://dx.doi.org/10.1145/2808117.2808120.
[7] D. Wu, R.K.C. Chang, W. Li, E.K.T. Cheng, D. Gao, MopEye: Opportunistic monitoring of per-app mobile network performance, in: Proceedings of USENIX ATC '17, USENIX Association, 2017, pp. 445–457.
[8] A. Razaghpanah, N. Vallina-Rodriguez, S. Sundaresan, C. Kreibich, P. Gill, M. Allman, V. Paxson, Haystack: In situ mobile traffic analysis in user space, 2015, CoRR URL http://arxiv.org/abs/1510.01419.
[9] Study on Encrypted Traffic Detection and Verification, Technical Specification (TS) 23.787, 3rd Generation Partnership Project (3GPP), 2018, Version 16.0.0.
[10] Certificate pinning, 2017, https://www.symantec.com/content/dam/symantec/docs/white-papers/certificate-pinning-en.pdf.
[11] M. Lotfollahi, M.J. Siavoshani, R.S.H. Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, Soft Comput. 24 (3) (2020) 1999–2012.
[12] Z. Hu, L. Zhu, J. Heidemann, A. Mankin, D. Wessels, P.E. Hoffman, Specification for DNS over Transport Layer Security (TLS), RFC 7858, 2016, http://dx.doi.org/10.17487/RFC7858, URL https://www.rfc-editor.org/info/rfc7858.
[13] G. Aceto, D. Ciuonzo, A. Montieri, A. Pescapé, Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges, IEEE Trans. Netw. Serv. Manag. 16 (2) (2019) 445–458, http://dx.doi.org/10.1109/TNSM.2019.2899085.
[14] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics (ISI), 2017, pp. 43–48.
[15] M.A. Hoque, M. Siekkinen, J.K. Nurminen, M. Aalto, S. Tarkoma, Mobile multimedia streaming techniques: QoE and energy saving perspective, Pervasive Mob. Comput. 16 (2015) 96–114, http://dx.doi.org/10.1016/j.pmcj.2014.05.004, URL http://www.sciencedirect.com/science/article/pii/S1574119214000807.
[16] Android AudioManager, 2020, https://developer.android.com/reference/android/media/AudioManager.
[17] Android TelephonyManager, 2020, https://developer.android.com/reference/android/telephony/TelephonyManager.
[18] Android CameraManager, 2020, https://developer.android.com/reference/android/hardware/camera2/CameraManager.
[19] A. Le, J. Varmarken, S. Langhoff, A. Shuba, M. Gjoka, A. Markopoulou, AntMonitor: A system for monitoring from mobile devices, in: Proceedings of C2B(1)D '15, ACM, 2015, pp. 15–20.
[20] A. Rämö, H. Toukomaa, Voice quality characterization of IETF opus codec, in: INTERSPEECH, 2011.
[21] Voice over IP - per call bandwidth consumption, 2016, https://www.cisco.com/c/en/us/support/docs/voice/voice-quality/7934-bwidth-consume.html.
[22] A. Schwind, F. Wamser, T. Gensler, P. Tran-Gia, M. Seufert, P. Casas, Streaming characteristics of spotify sessions, in: 2018 Tenth International Conference on Quality of Multimedia Experience (QoMEX), 2018, pp. 1–6, http://dx.doi.org/10.1109/QoMEX.2018.8463372.
[23] M.A. Hoque, A. Rao, S. Tarkoma, Network and application performance measurement challenges on android devices, SIGMETRICS Perform. Eval. Rev. 48 (3) (2021) 6–11, http://dx.doi.org/10.1145/3453953.3453955.
[24] X. Zhu, R. Pan, NADA: A unified congestion control scheme for low-latency interactive video, in: 2013 20th International Packet Video Workshop, 2013, pp. 1–8.
[25] G. Carlucci, L. De Cicco, S. Holmer, S. Mascolo, Analysis and design of the google congestion control for web real-time communication (WebRTC), in: Proceedings of the 7th International Conference on Multimedia Systems, MMSys '16, Association for Computing Machinery, New York, NY, USA, 2016, http://dx.doi.org/10.1145/2910017.2910605.
[26] Monsoon power monitor, 2020, https://www.msoon.com.
[27] J.C. Mogul, TCP offload is a dumb idea whose time has come, in: Proceedings of the 9th Conference on Hot Topics in Operating Systems - Volume 9, HOTOS '03, USENIX Association, Berkeley, CA, USA, 2003, p. 5, URL http://dl.acm.org/citation.cfm?id=1251054.1251059.
[28] N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, et al., Captum: A unified and generic model interpretability library for pytorch, 2020, arXiv preprint arXiv:2009.07896.
[29] B. Han, Mobile immersive computing: Research challenges and the road ahead, IEEE Commun. Mag. 57 (10) (2019) 112–118, http://dx.doi.org/10.1109/MCOM.001.1800876.
[30] A. Schulman, V. Navda, R. Ramjee, N. Spring, P. Deshpande, C. Grunewald, K. Jain, V.N. Padmanabhan, Bartendr: A practical approach to energy-aware cellular data scheduling, in: Proceedings of the Sixteenth Annual International Conference on Mobile Computing and Networking, MobiCom '10, ACM, New York, NY, USA, 2010, pp. 85–96, http://dx.doi.org/10.1145/1859995.1860006, URL http://doi.acm.org/10.1145/1859995.1860006.

[31] F. Xu, Y. Liu, T. Moscibroda, R. Chandra, L. Jin, Y. Zhang, Q. Li, Optimizing background email sync on smartphones, in: Proceeding of the 11th Annual International Conference on Mobile Systems, Applications, and Services, MobiSys '13, ACM, New York, NY, USA, 2013, pp. 55–68, http://dx.doi.org/10.1145/2462456.2464444, URL http://doi.acm.org/10.1145/2462456.2464444.
[32] 3GPP, System Architecture for 5G, Technical Specification (TS) 23.501, 3rd Generation Partnership Project (3GPP), 2017, Version 15.0.0.
[33] R. Ferrus, O. Sallent, J. Perez-Romero, R. Agusti, On 5G radio access network slicing: Radio interface protocol features and configuration, IEEE Commun. Mag. 56 (5) (2018) 184–192, http://dx.doi.org/10.1109/MCOM.2017.1700268.
[34] S. Kanugovi, F. Baboescu, J. Zhu, J. Mueller, S. Seo, Multiple Access Management Services Multi-Access Management Services (MAMS), RFC 8743, 2020, http://dx.doi.org/10.17487/RFC8743, URL https://www.rfc-editor.org/info/rfc8743.
[35] T. Mangla, E. Halepovic, M. Ammar, E. Zegura, MIMIC: Using passive network measurements to estimate HTTP-based adaptive video QoE metrics, in: 2017 Network Traffic Measurement and Analysis Conference (TMA), 2017, pp. 1–6, http://dx.doi.org/10.23919/TMA.2017.8002920.
[36] V. Krishnamoorthi, N. Carlsson, E. Halepovic, E. Petajan, BUFFEST: Predicting buffer conditions and real-time requirements of http(s) adaptive streaming clients, in: Proceedings of the 8th ACM on Multimedia Systems Conference, MMSys '17, ACM, New York, NY, USA, 2017, pp. 76–87, http://dx.doi.org/10.1145/3083187.3083193, URL http://doi.acm.org/10.1145/3083187.3083193.
[37] D. Bonfiglio, M. Mellia, M. Meo, D. Rossi, P. Tofanelli, Revealing skype traffic: When randomness plays with you, SIGCOMM Comput. Commun. Rev. 37 (4) (2007) 37–48, http://dx.doi.org/10.1145/1282427.1282386, URL http://doi.acm.org/10.1145/1282427.1282386.
[38] A. Reed, M. Kranch, Identifying HTTPS-protected netflix videos in real-time, in: Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy, CODASPY '17, ACM, New York, NY, USA, 2017, pp. 361–368, http://dx.doi.org/10.1145/3029806.3029821, URL http://doi.acm.org/10.1145/3029806.3029821.
[39] S. Kumar, S. Dharmapurikar, F. Yu, P. Crowley, J. Turner, Algorithms to accelerate multiple regular expressions matching for deep packet inspection, SIGCOMM Comput. Commun. Rev. 36 (4) (2006) 339–350, http://dx.doi.org/10.1145/1151659.1159952, URL http://doi.acm.org/10.1145/1151659.1159952.
[40] M. Roughan, S. Sen, O. Spatscheck, N. Duffield, Class-of-service mapping for QoS: A statistical signature-based approach to IP traffic classification, in: Proceedings of the 4th ACM SIGCOMM Conference on Internet Measurement, IMC '04, ACM, New York, NY, USA, 2004, pp. 135–148, http://dx.doi.org/10.1145/1028788.1028805.
[41] K. Suh, D.R. Figueiredo, J. Kurose, D. Towsley, Characterizing and detecting relayed traffic: A case study using Skype, in: IEEE Infocom, IEEE, 2006.
[42] P. Branch, L.H. Do, Real Time VoIP Traffic Classification, Tech. rep., Centre for Advanced Internet Architectures, Swinburne University of Technology, Melbourne, Australia, 2009.
[43] M.A. Hoque, P. Nurmi, M. Siekkinen, P. Hui, S. Tarkoma, The bits of silence: Redundant traffic in VoIP, in: MMSys '20, ACM, New York, NY, USA, 2020, http://dx.doi.org/10.1145/3339825.3391854.
[44] F. Li, J.W. Chung, M. Claypool, Silhouette: Identifying YouTube video flows from encrypted traffic, in: Proceedings of the 28th ACM SIGMM Workshop on Network and Operating Systems Support for Digital Audio and Video, NOSSDAV '18, ACM, New York, NY, USA, 2018, pp. 19–24, http://dx.doi.org/10.1145/3210445.3210448, URL http://doi.acm.org/10.1145/3210445.3210448.
[45] W. Jiang, M. Gokhale, Real-time classification of multimedia traffic using FPGA, in: 2010 International Conference on Field Programmable Logic and Applications, 2010, pp. 56–63, http://dx.doi.org/10.1109/FPL.2010.22.
[46] H. Kim, K. Claffy, M. Fomenkov, D. Barman, M. Faloutsos, K. Lee, Internet traffic classification demystified: Myths, caveats, and the best practices, in: Proceedings of the 2008 ACM CoNEXT Conference, CoNEXT '08, ACM, New York, NY, USA, 2008, pp. 11:1–11:12, http://dx.doi.org/10.1145/1544012.1544023, URL http://doi.acm.org/10.1145/1544012.1544023.